

R3.04 – Qualité de développement

2 – Bases du langage Java

Sébastien Thon

BUT Informatique 2ème année
IUT d'Aix-Marseille Université, site d'Arles
Département Informatique

1. Variables

Identifiant

Mêmes contraintes qu'en C++ (ne peut pas commencer par un chiffre ; distinction minuscules/majuscules, ...) ; mais on peut utiliser des caractères accentués.

Types simples

Ces types sont dits simples, car, à un instant donné, une variable de type simple ne peut contenir qu'une et une seule valeur.

Il y a 4 catégories :

1) Catégorie logique :

boolean

Deux états : **true** ou **false**.

2) Catégorie caractère :

String : chaîne de caractères.

char : caractère isolé. Codé en UNICODE sur deux octets → permet de représenter plusieurs milliers de caractères.

3) Catégorie entier

Quatre types d'entiers, forcément signés (« signed » et « unsigned » de C++ n'existent pas en Java) :

byte : 1 octet, de -128 à 127

short : 2 octets, de -32768 à 32767

int : 4 octets de -2 147 483 648 à 2 147 483 647

long : 8 octets de -9 223 372 036 854 775 808
à 9 223 372 036 854 775 807.

4) Catégorie réel

float : 4 octets, de $1.40239846 \times 10^{-45}$ à $3.40239846 \times 10^{38}$

double : 8 octets, de $4.94065645841246544 \times 10^{-324}$ à $1.79769313486231570 \times 10^{308}$

En Java, toute valeur réelle est définie par défaut en double précision (type **double**). Par conséquent, la lettre **d** (ou **D**) placée en fin de valeur n'est pas nécessaire.

```
double a = 3.14d;    // ça marche
```

```
double b = 3.14;     // ça marche aussi
```

Par contre, dès que l'on utilise une variable **float**, la lettre **f** (ou **F**) est indispensable, sous peine d'erreur de compilation.

```
float c = 3.14;      // ERREUR
```

```
float d = 3.14f;     // ça marche
```

Déclaration de variable

```
int test;
```

```
char choix, temp;
```

Déclaration de constante

On place le mot clé **final** devant la déclaration de la variable :

```
final int test = 10;
```

```
test = 2; // ERREUR
```

```
final int test2;
```

```
test2 = 2; // ça marche
```

```
Test2 = 3 ; // ERREUR
```

Casting

Java est un langage **fortement typé**.

Pour affecter un résultat d'un certain type dans une variable d'un autre type, il peut être nécessaire de faire un « **cast** » en plaçant entre parenthèse le nouveau type devant la variable à convertir.

Exemple : Affectation :

```
int    x;  
float  y;
```

```
x = y;
```

Produit une erreur lors de la compilation, car il n'est pas possible de mettre un `float` dans un `int` (perte de précision).

Solution : faire un cast de `y` en `int` avant de l'affecter à `x` :

```
x = (int)y;
```


Exemple: Diviser deux entiers :

```
int    x=7, y=2;  
float  res1, res2;
```

```
res1 = x/y;           // → res1 = 3
```

C'est une division entre entiers, qui donne un résultat entier

```
res2 =(float)x/y;     // → res2 = 3.5
```

x est convertit en `float`, ce qui permet de faire une division réelle

```
res2 =(float) (x/y) ;  // → res2 = 3
```

Attention, `x/y` donne un résultat entier, qui est ensuite casté en `float`

Exemple : Résultat d'une fonction :

```
float res;
```

```
res = Math.sqrt(4);
```

→ Produit une erreur lors de la compilation, car `Math.sqrt()` est une fonction retournant un `double`, et non pas un `float`. Il est possible de mettre un `float` dans un `double`, mais pas l'inverse (perte de précision).

→ Solution : on cast le résultat de `Math.sqrt()` en `float` :

```
float res;
```

```
res = (float)Math.sqrt(4);
```

2. Affichage en mode texte

La bibliothèque **System** contient des fonctionnalités permettant de réaliser des opérations d'entrées-sorties.

Par exemple, l'affichage de valeurs ou de texte dans la console est réalisé par l'utilisation de `System.out.print()`

Exemple :

```
System.out.print("Un peu de texte");
```

Exemples :

```
int valeur = 22;
```

```
System.out.print(valeur) ;
```

```
int var1=5, var2=10;
```

```
System.out.print("la premiere variable vaut " + var1  
                  + " et la seconde vaut " + var2 ) ;
```

```
System.out.print ("Il fait beau");
```

```
System.out.print (" et chaud");
```

→ Ne saute pas de ligne :

```
Il fait beau et chaud
```

```
System.out.println("Il fait beau");
```

```
System.out.print (" et chaud");
```

```
Il fait beau  
et chaud
```

3. Saisie de données au clavier

Pour saisir une valeur au clavier dans un environnement non graphique, on peut utiliser la classe **Scanner** du package **java.util**

Cette classe dispose de méthodes de lecture pour chaque type simple :

```
Scanner lectureClavier = new Scanner(System.in);  
byte octet = lectureClavier.nextByte();  
int entier = lectureClavier.nextInt();  
float réel = lectureClavier.nextFloat();  
String mot = lectureClavier.next();  
char caractère = lectureClavier.next().charAt(0);  
String phrase = lectureClavier.nextLine();
```

Exemple

```
import java.util.*;

public class Saisie
{
    public static void main (String [] arg)
    {
        int    entier;
        String phrase;
        Scanner lectureClavier = new Scanner(System.in) ;

        System.out.println("Entrez un entier : ");
        entier = lectureClavier.nextInt();

        lectureClavier.nextLine();
        System.out.println("Entrez une phrase : ");
        phrase = lectureClavier.nextLine();
    }
}
```

4. Commentaires

Les commentaires sont les mêmes qu'en C++

```
/* Voici un commentaire  
   sur plusieurs lignes.  
*/
```

```
int nom; // Commentaire
```


5. Conditions

La syntaxe des conditions, des opérateurs logiques, est la même qu'en C++. On peut aussi utiliser l'instruction switch.

```
if (condition1)
{
    instruction1;
}
else if (condition2 && condition3)
{
    instruction2;
}
else
{
    instruction3;
}
```

```
switch (valeur)
{
    case etiquette1 :
        // une ou plusieurs instructions
        break;

    case etiquette2 :
    case etiquette3 :
        // Une ou plusieurs instructions
        break;

    default :
        // Une ou plusieurs instructions
}
```

6. Boucles

On retrouve les mêmes types de boucles qu'en C++ :

- La boucle **répéter ... tant que**

```
do
{
    instructions
} while (condition);
```

- La boucle **tant que ...**
while () ...

```
while (condition)
{
    instructions
}
```

- La boucle **répéter pour...**
for () ...

```
for (initialisation ; condition ; incrément)
{
    instructions
}
```

7. La classe Math

<https://docs.oracle.com/en/java/javase/16/docs/api/java.base/java/lang/Math.html>

Elle se trouve dans le **package** java.lang qui est importé par défaut.

Elle contient un ensemble de fonctions mathématiques (cosinus, sinus, racine carrée, puissance, ...), sous forme de **méthodes statiques**.

→ ce n'est pas la peine d'instancier la classe sous la forme d'un objet pour en appeler les méthodes.

Le nom de chaque méthode débute par **Math**, suivi d'un point, puis du nom de la fonction.

Quelques fonctions

On définit les variables :

```
double resultat, angle, a, b;
```

- Cosinus (en radian) d'un angle : **Math.cos()**

Ex: `resultat = Math.cos(angle);`

- Sinus (en radian) d'un angle : **Math.sin()**

- Logarithme : **Math.log()**

Ex: `resultat = Math.log(a);`

- Exponentielle : **Math.exp()**

Ex: `resultat = Math.exp(a);`

- Racine carré d'un nombre : **Math.sqrt()**
Ex: `resultat = Math.sqrt(angle);`
- a^b : **Math.pow()**
Ex: `resultat = Math.pow(a,b);`
- $|a|$ (valeur absolue de a) : **Math.abs()**
Ex: `resultat = Math.abs(a);`
- Tirer un nombre au hasard entre 0 et 1 : **Math.random()**
Ex: `resultat = Math.random();`
- Minimum de deux nombres : **Math.min()**
Ex: `resultat = Math.min(a,b);`
- Maximum de deux nombres : **Math.max()**

8. Ecrire une méthode

Contrairement au C++, tout le code d'une classe est rangé dans un fichier .java, il n'y a pas de partage entre un fichier d'include (.h) qui contiendrait des prototypes et un fichier qui contiendrait l'implémentation des méthodes (.cpp)

```

public class Cercle
{
    // Méthode principale
    public static void main( String [] arg )
    {
        Scanner clavier = new Scanner(System.in);
        double valeur, résultat;
        System.out.print("Valeur du rayon : ");
        valeur = clavier.nextDouble();
        résultat = périmètre (valeur);
        System.out.println("Périmètre = " + résultat);
    } // fin de main()

    // méthode périmètre
    public static double périmètre (double rayon)
    {
        double p;
        p = 2 * Math.PI * rayon;
        return p;
    } // fin de périmètre()
} //fin de class Cercle

```

Les différentes formes d'une méthode

- Méthode avec résultat

Voir la méthode `périmètre()`. Le résultat était un double. Ce type doit être annoncé dans le prototype de la méthode :

```
public static double périmètre (double rayon)
```


- Méthode sans résultat

Certaines méthodes ne renvoient pas de résultat.

```
public static void affiche (int valeur)
{
    System.out.print(valeur) ;
}
```

void signifie que la méthode **affiche()** ne retourne pas de résultat. Dans ce cas, il ne doit pas y avoir de **return** dans le corps de la méthode.

- Méthode sans paramètre

Une méthode peut ne pas avoir de paramètre.

```
public static void affiche_bonjour()  
{  
    System.out.print("Bonjour") ;  
}
```

- Méthode à plusieurs paramètres

Si nous écrivons par exemple une méthode `max()` qui permet de déterminer le plus grand de deux entiers, il y a deux paramètres, `a` et `b` de type entier :

```
public static int max (int a, int b)
{
    if( a > b )
        return a;
    else
        return b;
}
```

Les paramètres sont séparés par une **virgule**. Devant **chaque** paramètre, il faut indiquer son **type**.

Attention

Les types simples (int, short, float, double, char, boolean) sont toujours passés par **valeur** à une méthode (→ la méthode ne peut pas en modifier le contenu)

Les objets sont toujours passés par **référence** à une méthode (→ la méthode peut en modifier le contenu)

Le passage par **adresse** n'existe pas (il n'y a d'ailleurs pas de pointeur en java).